DEEP CALIBRATION OF STOCHASTIC VOLATILITY MODELS VIA NEURAL SURROGATES

A Project Report Submitted

for the Course

MA4998P Project I

by

Raj Nandani Kumari (Roll No. 230123048)

Saanvi Goel (Roll No. 230123054)



to the

DEPARTMENT OF MATHEMATICS

INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

GUWAHATI - 781039, INDIA

November 2026

CERTIFICATE

This is to certify that the work contained in this project report entitled Deep Calibration of Stochastic Volatility Models via Neural Surrogates submitted by Saanvi Goel (Roll No.: 230123054) and Raj Nandani Kumari (Roll No.: 230123048) to the Department of Mathematics, Indian Institute of Technology Guwahati towards partial requirement of Bachelor of Technology in Mathematics and Computing has been carried out by him/her under my supervision.

It is also certified that this report is a survey work based on the references in the bibliography.

Turnitin Similarity: _6_ % and AI content: _< 20_ %

Guwahati - 781039
November 2026

Saanvi Goel and Raj Nandani Kumari
Project Student(s)

Guwahati - 781039
November 2026

Prof. Satyajit Pramanik
Project Supervisor

ABSTRACT

Calibrating parameters of stochastic volatility models via classical numerical quadrature requires iterative root-finding, creating a latency bottleneck in derivative pricing. This work develops an arbitrage-free, mathematically regularized Deep Neural Network forward surrogate for the Heston model, delivering millisecond evaluation speeds. We generate our ground-truth implied volatility surfaces using stratified Latin Hypercube Sampling with single-integral Fourier inversion. The framewrok is validated against QuantLib, achieving an absolute error strictly below 10^{-4}.

The two issues in existing random-grid pointwise benchmarks are that there is an unintended five-layer loop artifact and the input features are unscaled. We made three changes. The first change is that we used dimensionless log-moneyness parameterization to ensure degree-1 scale homogeneity. Second change is affine min-max normalization which maps input features to $[-1, 1]^7$. This improves Hessian conditioning. Final change is to adopt a four-layer ELU architecture and maintain C^1 Sobolev smoothness.

In controlled A/B experiments, these changes reduced test RMSE by 66.94% and model parameters by 24.2%, resulting in a network with 13,057 weights. We evaluated the surrogate on 10,010 contracts across 70 configurations. On a strictly held-out 15% test set, the model achieved an R^2 of 0.999408, RMSE of 0.001878, and MAE of 0.001351. Diagnostic audits verify numerical stability down to 1.8-day tenors and under extreme Feller condition violations ($2\kappa\theta/\sigma^2 = 0.222$), show autograd Jacobian agreement with finite differences within 9.59×10^{-5}, and establish zero calendar or butterfly arbitrage violations across an 875-contract grid. Future work will focus on integrating this forward surrogate into an end-to-end inverse calibration pipeline.

Contents

List of Figures **vii**

List of Tables **viii**

List of Abbreviations **ix**

1 Introduction and Problem Formulation **1**

1.1 The Quantitative Finance Landscape and Volatility Dynamics 2

1.2 Breakdown of the Black-Scholes Framework 2

1.3 The Heston Stochastic Volatility Model and Financial Parameters 4

1.4 The Calibration Bottleneck in Options Pricing 6

1.5 Proposed Machine Learning Framework and Project Objectives 7

1.6 Organization of the Report 9

2 Evolution of Neural Calibration Paradigms **10**

2.1 The Direct Inverse Approach 10

2.2 The Two-Stage Grid-Based Paradigm 11

2.3 The Pure Pointwise Approach 12

2.4 The Random-Grid Pointwise Formulation 12

3 Mathematical Framework and Pricing Engines **14**

3.1 Parameter Space Sampling via Latin Hypercube Sampling (LHS) 14

3.2 Stratified Tenors and Asymmetric Adaptive Strikes 15

3.3 Semi-Analytical Heston Pricing via Fourier Inversion 16

3.4 Branch-Cut Discontinuities and the Little Heston Trap 17

3.5 Boundary Behaviors and the CIR Feller Condition 18

3.6 Black-Scholes Implied Volatility Inversion via Brent's Method 18

3.7 Data Generation and C++ QuantLib Validation Benchmark . 19

4 Repository Audit and Architectural Refinement **21**

4.1 Architectural Discrepancy and Parameter Bloat in the Benchmark Codebase 21

4.2 Scale Homogeneity and Input Normalization 23

4.2.1 Log-Moneyness as a Scale-Free Input 24

4.2.2 Dynamic MinMax Affine Normalization 25

4.3 Network Topology, Differentiability, and Approximation Capacity 26

4.3.1 Mathematical Formulation of the Network 26

4.3.2 Universal Approximation and Model Capacity 27

4.3.3 C^1 Sobolev Smoothness via ELU Activations 27

5 Empirical Experiments and Comparative Benchmarks **30**

5.1 Controlled A/B Convergence Race 30

5.2 Gradient Stability and Accelerated Convergence 32

5.3 Scaling the Synthetic Training Library 33

5.4 Test Generalization and the 45-Degree Parity Diagnostic . . . 34

5.5 Multi-Maturity Implied Volatility Smile Overlays 36

5.5.1 Short-Maturity Volatility Smiles 36
5.5.2 Long-Maturity Volatility Smiles 38
5.6 Summary of Experimental Findings 39

6 Mathematical and Physical Diagnostic Audits 40
6.1 Singularity Audit: Short Maturity and Feller Condition Violation 40
6.2 Gradient Validation: Autograd Against Finite Differences . . . 41
6.3 Static No-Arbitrage Verification 42
6.3.1 Calendar Spread Monotonicity 42
6.3.2 Butterfly Arbitrage and Risk-Neutral Density Positivity 43
6.4 Additional Numerical Results 44

7 Conclusion and Future Work 45
7.1 Conclusion 45
7.2 Future Work: Phase 2 46

Bibliography 48

List of Figures

5.1 Validation RMSE during training for the baseline and refined networks. 32
5.2 Distributions of the generated contracts across maturity, strike, and implied volatility. 34
5.3 Predicted implied volatilities plotted against the Fourier reference values. The dashed line indicates perfect agreement. . . 35
5.4 Surrogate predictions compared with the Fourier reference smiles at maturities of 0.05 and 0.25 years. 37
5.5 Heston implied volatility as a function of strike at 0.25 years. . 37
5.6 Surrogate predictions compared with the Fourier reference smiles at maturities of 1.0 and 2.0 years. 38

6.1 Fourier integrand decay for maturities between 1.8 days and 1y. 41
6.2 Total implied-variance surface and numerical call-price curvature diagnostics. 43

List of Tables

1.1 The Five Heston Parameters and Physical Interpretation . . . 5

2.1 Neural Calibration Paradigm Comparison 13

4.1 Comparison of baseline and refined neural network architectures 23

5.1 Controlled A/B comparison of validation RMSE 31
5.2 Synthetic dataset generation summary 33
5.3 Performance of the refined surrogate on the held-out test set . 35

6.1 Discrete no-arbitrage checks on the 875-contract evaluation mesh. 44
6.2 Autograd versus central finite-difference parameter sensitivities. 44

List of Abbreviations

CIR	Cox-Ingersoll-Ross
CPU	Central Processing Unit
DNN	Deep Neural Network
ELU	Exponential Linear Unit
GBM	Geometric Brownian Motion
LHS	Latin Hypercube Sampling
MAE	Mean Absolute Error
MLP	Multi-Layer Perceptron
RMSE	Root Mean Squared Error
SDE	Stochastic Differential Equation

Chapter 1

Introduction and Problem Formulation

The pricing and risk management of financial derivatives rely heavily on mathematical models that can capture the behaviour of financial markets. Traditional models, such as Black–Scholes, provide a useful starting point but assume constant volatility, which does not fully reflect real market conditions. Heston model is a stochastic volatility model that can address this limitation as it allows volatility to change over time.

However, these models can be computationally expensive to calibrate using conventional numerical methods. This creates a challenge when fast and repeated evaluations are required. As an alternative to traditional pricing methods, We have therefore explored the use of deep neural networks. A neural network can learn the relationship between model parameters and implied volatility. It provides a faster approximation and also retains good predictive accuracy.

1.1 The Quantitative Finance Landscape and Volatility Dynamics

In quantitative finance, derivatives pricing, hedging, and risk management depend on understanding how financial markets behave over time. Value of an equity or an index option is dependent on the price movements of its underlying asset which is represented by S_t from the time 0 to its maturity T. Volatility is also important because it affects both the value of an option and the way its risk changes with market conditions. In practice, traders and risk managers use implied volatility to compare option prices across different strikes and maturities. The resulting volatility surface provides useful information about how the market views uncertainty over different time horizons. Understanding its shape is therefore an important part of option pricing and risk management.

1.2 Breakdown of the Black-Scholes Framework

The foundational Black-Scholes-Merton model [5] assumes that the underlying asset price follows a Geometric Brownian Motion (GBM) with constant risk-free drift r and constant instantaneous volatility σ:

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t \tag{1.1}$$

Under this model, the price of a European call option with strike price K and maturity T is given by:

$$C_{\mathrm{BS}}(S_0, K, T, \sigma) = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{1.2}$$

where

$$d_1 = \frac{\ln(S_0/K) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T} \tag{1.3}$$

and $\Phi(\cdot)$ represents the cumulative distribution function of the standard normal distribution. If the assumptions of the Black-Scholes model held in real markets, implied volatility would be the same for options with different strike prices and maturities. The implied volatility surface $\sigma_{\mathrm{iv}}(K, T)$ would therefore be flat and equal to σ. However, market prices show that this is not usually the case. Implied volatility changes with the strike price. This forms patterns known as *volatility smiles* and *volatility skews*. In a volatility smile, options that are far from the at-the-money level tend to have higher implied volatility. A volatility skew is slightly different. In equity and index markets, out-of-the-money put options often have higher implied volatility because investors may be willing to pay more for protection against a market fall. These patterns show two features of financial markets that the Black–Scholes model cannot fully explain:

1. **Fat Tails and Excess Kurtosis:** Financial returns can show larger movements than those predicted by a log-normal model. Large losses, in particular, may occur more often than the model suggests.

2. **Dynamic Leverage Effect:** Volatility often rises when equity prices fall. One reason is that a fall in a company's share price increases its financial leverage. As a result, negative returns are often associated

with an increase in volatility.

These differences matter when pricing options. A model with constant volatility may not match market prices across all strikes and maturities, especially for options that are far from at-the-money. The problem is that volatility in real markets changes over time. This is one reason to consider stochastic volatility models, which allow volatility to evolve and can better represent its relationship with the underlying asset price [9].

1.3 The Heston Stochastic Volatility Model and Financial Parameters

Stochastic volatility models allow the variance of the underlying asset to change randomly over time. This resolves the limitations of constant-volatility models. Two-factor Heston model, introduced by Heston (1993) [9] is widely used in quantitative finance. Under the risk-neutral measure $\mathbb{Q}$, the asset price S_t and its instantaneous variance v_t follow the coupled stochastic differential equations (SDEs):

$$dS_t = rS_t\,dt + \sqrt{v_t}S_t\,dW_t^S \tag{1.4}$$

$$dv_t = \kappa(\theta - v_t)\,dt + \sigma\sqrt{v_t}\,dW_t^v \tag{1.5}$$

where W_t^S and W_t^v are standard Brownian motions with continuous correlation $d\langle W^S, W^v\rangle_t = \rho\,dt$ for $\rho \in [-1, 1]$.

The model dynamics are parameterized by $\mathbf{\Theta} = (v_0, \theta, \kappa, \sigma, \rho)$ as summarized in Table 1.1:

The correlation parameter ρ helps capture the relationship between changes in the asset price and changes in volatility, while the mean-reverting variance

Table 1.1: The Five Heston Parameters and Physical Interpretation

Parameter	**Symbol**	**Financial Meaning & Physical Role**
Initial Variance	v_0	Current variance (current market turbulence)
Long-term Variance	θ	Long-term average variance (baseline variance toward which v_t reverts)
Mean-Reversion Speed	κ	Speed of mean reversion (rate at which variance returns to baseline)
Vol-of-Vol	σ	Volatility of variance (dictating tail kurtosis)
Correlation	ρ	Asset-volatility correlation (leverage effect, generating skew)

process provides a way to model how volatility evolves around its long-term level. So, the model is useful for studying implied volatility smiles and skews because of these features. And the mathematical structure allows option prices to be evaluated using characteristic functions and numerical integration [9].

The variance process follows a square-root Cox-Ingersoll-Ross (CIR) diffusion (Cox et al., 1985 [6]). For the variance process to remain strictly positive ($v_t > 0$) almost surely, the model parameters must satisfy the Feller condition (Feller, 1951 [7]):

$$2\kappa\theta > \sigma^2 \tag{1.6}$$

In our setup to generate synthetic data, we have set initial spot $S_0 = 100.0$, risk-free rate $r = 0.0$, and continuous dividend yield $q = 0.0$. This is consistent with standard experimental benchmarks.

1.4 The Calibration Bottleneck in Options Pricing

A stochastic model must be calibrated to match observable exchange-traded vanilla option quotes before it can price exotic contracts or compute portfolio Greeks. Calibration is formulated as an inverse non-linear least-squares optimization problem seeking the parameter vector $\hat{\boldsymbol{\Theta}}$ that minimizes the distance between model-implied and market-observed implied volatilities over a collection of strikes and maturities:

$$\hat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta}\in\mathcal{D}} d\left(\sigma_{\mathrm{BS}}^{\mathcal{M}}(\boldsymbol{\Theta}, K, T),\, \sigma_{\mathrm{BS}}^{\mathrm{mkt}}(K, T)\right) \tag{1.7}$$

where the objective metric is typically the root-mean-square error (RMSE):

$$d(x, y) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\sigma_{\mathrm{BS}}^{\mathcal{M}}(\boldsymbol{\Theta}, K_i, T_i) - \sigma_{\mathrm{BS}}^{\mathrm{mkt}}(K_i, T_i)\right)^2} \tag{1.8}$$

and $\mathcal{D}$ denotes the admissible parameter hypercube.

Iterative optimizers (such as Levenberg–Marquardt, BFGS, or Nelder–Mead) propose consecutive candidate parameter combinations. For each candidate vector $\boldsymbol{\Theta}$, the pricing routine must evaluate semi-analytical Fourier inversion integrals across hundreds of contracts, invert each dollar price to Black–Scholes implied volatility via iterative root-finding, and approximate Jacobians using finite-difference stencils.

Classical Heston calibration can take even up to 10 to 30 seconds for a single surface slice, according to runtime estimates in the quantitative finance literature, but electronic market-making often requires updates in less than a second. If calibration takes too long, trading desks may continue to use

outdated parameters. This can make hedging less effective and increase the risk of adverse selection. For this reason, conventional calibration can be difficult to use in applications where market conditions change rapidly.
We can reduce this repeated computational effort by training a model that approximates the pricing relationship in advance. Deep neural networks can learn complex, non-linear relationships from generated data. They can also provide predictions for new inputs without running the full numerical pricing procedure each time after training them once. Earlier studies have used neural networks in option pricing and in similar computational tasks, and the results also suggest that they can serve as approximations to more expensive numerical methods [2]. However, their performance depends on several factors, including the quality of the training data, the choice of input variables and the network architecture. Low prediction error alone is not enough. The resulting model must also behave consistently and produce numerically reliable outputs.

1.5 Proposed Machine Learning Framework and Project Objectives

Heston calibration is computationally expensive. So this project explores a two-phase framework based on deep neural networks. In first phase we focused on learning the relationship between the model parameters, option characteristics, and implied volatility and the framework for that is designed as follows:

1. **Offline Learning (Phase 1):** A deep neural network is trained offline. The mapping from Heston model parameters and option characteristics

to implied volatility is approximated:

$$\mathcal{NN}(\boldsymbol{\Theta}, T, K) \approx \sigma_{\text{iv}}(\boldsymbol{\Theta}, T, K) \tag{1.9}$$

2. **Online Calibration (Phase 2):** In the next phase we plan to extend the trained forward surrogate to inverse calibration. Network predictions and their gradients, computed through automatic differentiation would be used. This will be combined with optimization routine such as Levenberg-Marquardt. This would avoid repeatedly evaluating the computationally expensive Heston pricing integrals during calibration. This could substantially reduce evaluation time.

In this project we have also examined the numerical behaviour of the learned approximation. We have paid attention to the choice of input features, data normalization, and network architecture. These factors influence both prediction accuracy and the stability of derivatives obtained through automatic differentiation. Hence the Phase 1 work focuses on developing and evaluating a reliable forward surrogate before extending it to inverse calibration in Phase 2.

The specific objectives established for Phase 1 are: Implementing a semi-analytical pricing engine for European call options which can be used as a benchmark under Heston dynamics; Generating pointwise training synthetic data of the 5-dimensional Heston parameter space using LHS; Replicating the random-grid pointwise data generation idea described in Baschetti et al. (2024) [2]; Making refinements on it which involve layer count, dimensionless log-moneyness, and feature normalization; Evaluating predictive accuracy on the 15% held-out data; Examining numerical behavior at maturities which are close to 0; Checking differentiability and static no-arbitrage consistency

conditions.This all is done before proceeding to inverse calibration in Phase 2.

1.6 Organization of the Report

The rest of this report is organized into the following chapters:
Chapter 2 traces the development of neural network-based calibration methods and compares four approaches: direct, fixed-grid, pure pointwise, and random-grid formulations.
Chapter 3 establishes the mathematical ground-truth pricing baseline using LHS, detailing Lewis Fourier inversion, Albrecher branch-cut resolution, Brent's root-finding, and C++ QuantLib benchmarks.
Chapter 4 presents our engineering audit of the benchmark codebase (NNpnt.ipynb), detailing the structural flaws identified and the three mathematical refinements introduced.
Chapter 5 discusses our empirical A/B convergence experiments, dataset scaling to 10,010 contracts, parity diagnostics, and multi-maturity smile overlays.
Chapter 6 details the three diagnostic audits: short-maturity singularity analysis, Sobolev Autograd regularity, and static no-arbitrage certification.
Chapter 7 discusses the findings of this work, reviews the limitations and model artifacts identified during the study, and presents the roadmap for Phase 2, which will focus on inverse calibration using the Levenberg-Marquardt algorithm.

Chapter 2

Evolution of Neural Calibration Paradigms

Neural networks can approximate computationally expensive option pricing functions by learning the relationship between model parameters and option characteristics. However, the choice of training strategy and input representation plays an important role in the accuracy and generalization of the resulting surrogate. This chapter reviews four approaches to neural network-based calibration, discussing their development, advantages, and limitations. The leads to the random-grid pointwise framework used in this project.

2.1 The Direct Inverse Approach

An early application of neural networks to model calibration was presented by Hernandez (2017) [8], who explored the Hull-White interest rate model. The study used a direct inverse approach, in which a deep neural network was trained to estimate model parameters from market data. More specifically, the network took a normalized grid of market implied volatilities as input

and returned the corresponding optimal parameter vector $\hat{\boldsymbol{\Theta}}$.

$$\mathcal{NN}_{\text{direct}} \colon \mathbb{R}^{n \times m} \longrightarrow \boldsymbol{\Theta} \in \mathbb{R}^{p} \tag{2.1}$$

The direct approach is conceptually straightforward but has certain limitations: it may fail when market conditions move beyond its historical training data, and it lacks a forward pricing mechanism for evaluating custom option contracts or calculating risk-management hedges.

2.2 The Two-Stage Grid-Based Paradigm

In this approach the neural network first predicts implied volatilities at a fixed set of points. These points are predefined for given model parameters. These predictions are then used to find the parameters by an optimizer. These parameters are the best match for the observed market data. This approach restores the forward pricing functional. In Horvath et al. (2021) [11], a feedforward neural network with 4 hidden layers and 30 nodes per layer maps model parameters $\boldsymbol{\Theta}$ to an 8×11 tensor grid of implied volatilities (8 maturities $\times$ 11 strikes = 88 output pixels). Training on 68,000 to 80,000 synthetic surfaces allowed an optimizer such as Levenberg–Marquardt to calibrate market surfaces in just a few milliseconds:

$$\rho_{\text{grid}} \colon \boldsymbol{\Theta} \in \mathbb{R}^{p} \longrightarrow \sigma_{\text{grid}} \in \mathbb{R}^{8 \times 11} \tag{2.2}$$

Despite these speedups, fixed grids have certain limitations: they may poorly cover very short maturities ($T < 0.1$ years), requiring interpolation or extrapolation, and fixed strike prices cannot automatically adjust as option expiration dates change.

2.3 The Pure Pointwise Approach

Bayer and Stemper (2018) and Liu et al. (2019) [4, 13] explored an alternative approach in which the network predicts implied volatility for one option contract at a time. It takes the model parameters and contract characteristics as inputs and returns a single implied volatility value.

$$\mathcal{NN}_{\text{pointwise}} \colon (\boldsymbol{\Theta}, T, K) \in \mathbb{R}^{p+2} \longrightarrow \sigma_{\text{iv}} \in \mathbb{R} \tag{2.3}$$

A predefined grid is not needed here. This is because the network can evaluate arbitrary pairs (T, K) directly. This eliminates the need for interpolation and extrapolation. However, this approach can require very large training sets and expensive individual pricing calls. It may involve sampling 10^6 independent parameter tuples, each paired with a single strike-maturity pair, and generating a separate pricing result for each. This approach is computationally expensive because training on such datasets which are large and unstructured can require large networks. Large networks have 3 to 4 hidden layers of 4,096 nodes each.

2.4 The Random-Grid Pointwise Formulation

Introduced by Baschetti et al. (2024) [2], this formulation combines the data-generation efficiency of the grid approach with the interpolation-free flexibility of the pointwise approach:

$$[\boldsymbol{\Theta}, T, k] \longrightarrow [\text{Parsimonious 4-Layer MLP (64 nodes)}] \longrightarrow \sigma_{\text{iv}} \in \mathbb{R} \tag{2.4}$$

Random-grid training offers two main advantages over the pure pointwise

approach:

- **Computational Efficiency in Data Generation:** Semi-analytical pricing models rely on characteristic functions that depend on the model parameter vector $\boldsymbol{\Theta}$ and maturity T. Multiple strikes are evaluated along the same maturity slice. This can reduce dataset generation time by up to a factor of m which is equal to number of strikes; as compared to generating every coordinate triplet $(\boldsymbol{\Theta}, T, K)$ independently.

- **Structure of Volatility Smiles is learnt:** We are providing the network with structured data across multiple maturities for each parameter tuple. This helps it to learn how the implied volatility smile changes with strike and maturity. This improves its ability to capture patterns in the volatility surface compared to training on independent pointwise samples. This also contributes to smoother predictions, more stable convergence, and better performance on unseen data.

Table 2.1 summarizes the main differences between the four neural network-based calibration approaches.

Table 2.1: Neural Calibration Paradigm Comparison

Paradigm	**Input Space**	**Interpolation**	**Output Space**
Direct Inverse	Grid of volatilities $(N \times M)$	Yes, to match the grid	Parameter vector $(\boldsymbol{\Theta})$
Two-Stage Grid-Based	Parameter vector $(\boldsymbol{\Theta})$	Yes, 2D spline	Fixed volatility grid (8×11)
Pure Pointwise	Parameters and coordinates $(\boldsymbol{\Theta}, T, K)$	No	Scalar volatility (σ)
Random-Grid Pointwise	Parameters and coordinates $(\boldsymbol{\Theta}, T, K)$	No	Scalar volatility (σ)

Chapter 3

Mathematical Framework and Pricing Engines

For training the forward neural surrogate we require a dataset of synthetic option prices and their corresponding implied volatilities. To construct this dataset, we use a pricing and inversion pipeline designed to balance accuracy with efficiency and stability. This chapter describes the procedure used to generate the training data and the methods adopted to ensure its reliability.

3.1 Parameter Space Sampling via Latin Hypercube Sampling (LHS)

To ensure the neural surrogate covers a broad spectrum of market environments, training data must achieve uniform coverage across the continuous five-dimensional Heston parameter space: $\Theta = (\kappa, \theta, \sigma, \rho, \text{v_0}) \in \mathcal{D} \subset \mathbb{R}^5$. Hence we avoid standard Pseudo-Random Monte Carlo sampling which is a uniform random sampling because in that points tend to cluster locally. Instead we implement Latin Hypercube Sampling (LHS) (McKay, Beckman,

& Conover, 1979) [14]. LHS is a multidimensional stratified sampling technique. In LHS each of the five parameter intervals is divided into N equal probable, non-overlapping bins. One sample is drawn from each bin. The samples from different dimensions are randomly combined to form parameter vectors. This approach helps distribute the samples evenly across the parameter space.

Parameters follow standard institutional benchmarks ($S_0 = 100.0$, $r = 0.0$, $q = 0.0$) across operational ranges: $\kappa \in [0.5, 4.0]$, $\theta \in [0.01, 0.20]$, $\sigma \in [0.05, 0.80]$, $\rho \in [-0.90, -0.10]$, and $v_0 \in [0.01, 0.20]$. Each sample is checked for the Feller condition, i.e. the constraint ($2\kappa\theta - \sigma^2 > 0$). Only vectors that meet this condition are retained. This results in exactly 70 admissible parameter configurations.

3.2 Stratified Tenors and Asymmetric Adaptive Strikes

We sample option maturities and strike prices (T, K) for all parameter vectors $\boldsymbol{\Theta}$. The maturity range $[0.003, 2.5]$ years is divided into 11 subintervals and one maturity randomly selected from each interval. This sampling strategy ensures that both very short-term and longer-dated options are represented in the dataset. For each maturity, 13 strikes are selected using maturity-dependent adaptive bounds and the allocation is done as: four left-tail, seven central and two right-tail strikes.

Adaptive Strike Diffusion Boundaries:

$$[K_{\min}(T), K_{\max}(T)] = \left[S_0(1 - 0.55\sqrt{T}),\ S_0(1 + 0.30\sqrt{T})\right] \tag{3.1}$$

Non-Uniform Strike Allocation Intervals:

- Left-tail strikes (4 strikes): $K \in [K_{\min}(T),\, S_0(1 - 0.20\sqrt{T})]$, densely capturing out-of-the-money put skew and downside crash hedging.
- Central region strikes (7 strikes): $K \in [S_0(1-0.20\sqrt{T}),\, S_0(1+0.20\sqrt{T})]$, resolving the at-the-money curvature.
- Right-tail strikes (2 strikes): $K \in [S_0(1+0.20\sqrt{T}),\, K_{\max}(T)]$, spanning out-of-the-money calls.

3.3 Semi-Analytical Heston Pricing via Fourier Inversion

In the Heston model, European call prices $C(S_0, K, T)$ are computed semi-analytically by inverting the characteristic function of the log-asset price. Implementing the single-integral formulation of Lewis (2001) [12] instead of the two separate probability integrals in Heston (1993) [9], reduces the numerical quadrature latency by 50% while preserving precision.

$$C(S_0, K, T) = S_0 - \frac{\sqrt{S_0 K}}{\pi} \int_0^\infty \mathrm{Re}\left[e^{-iuk_{\text{fwd}}} \frac{\phi\left(u - \frac{i}{2}\right)}{u^2 + \frac{1}{4}} \right] du \tag{3.2}$$

where:

- $k_{\text{fwd}} = \ln(S_0/K) + rT$ is the forward log-moneyness;
- u is the integration frequency variable;
- $\phi(\xi) = \mathbb{E}^{\mathbb{Q}}\left[\exp\left(i\xi \ln(S_T/S_0)\right)\right]$ is the generalized characteristic function of the log-asset price evaluated along the complex contour $\xi = u - i/2$.

3.4 Branch-Cut Discontinuities and the Little Heston Trap

Evaluating the characteristic function $\phi(\xi)$ involves solving a system of coupled Riccati ordinary differential equations. The classical Heston formulation involves a complex logarithm:

$$C(\xi, T) = ri\xi T + \frac{\kappa\theta}{\sigma^2}\left[(\kappa - i\rho\sigma\xi - d)T - 2\ln\left(\frac{1 - ge^{-dT}}{1 - g}\right)\right] \tag{3.3}$$

$$d = \sqrt{(\kappa - i\rho\sigma\xi)^2 + \sigma^2(\xi^2 + i\xi)} \tag{3.4}$$

$$g = \frac{\kappa - i\rho\sigma\xi - d}{\kappa - i\rho\sigma\xi + d} \tag{3.5}$$

The Little Heston Trap: The complex-valued quantity $1 - ge^{-dT}$ may cross the negative real axis in the complex plane $\mathbb{C}$ at large maturity T. This is problematic because the principal branch of the complex logarithm has a branch cut along $(-\infty, 0]$. Crossing this cut can introduce a discontinuous jump of $\pm 2\pi i$ which may lead to numerical instability and produce spikes. It can even produce negative option prices.

The Albrecher Resolution: Setting $g_* = 1/g$, Albrecher et al. (2007) [1] restructures the Riccati coefficients as:

$$C(\xi, T) = ri\xi T + \frac{\kappa\theta}{\sigma^2}\left[(\kappa - i\rho\sigma\xi - d)T - 2\ln\left(\frac{e^{dT} - g_*}{1 - g_*}\right)\right] \tag{3.6}$$

$$D(\xi, T) = \left(\frac{\kappa - i\rho\sigma\xi - d}{\sigma^2}\right)\left[\frac{e^{dT} - 1}{e^{dT} - g_*}\right] \tag{3.7}$$

Since $|g_*| > 1$ for all real integration frequencies u, the modified formulation avoids the branch-cut crossing which was seen earlier. This helps maintain the smoothness and continuity of the characteristic function across different maturities.

3.5 Boundary Behaviors and the CIR Feller Condition

The variance process v_t is modelled using a square-root CIR diffusion. When the Feller condition $2\kappa\theta > \sigma^2$ is satisfied, the drift pushes the process away from zero, preventing it to reach zero in finite time. When $2\kappa\theta \leq \sigma^2$, the origin becomes an accessible, reflecting boundary where variance touches zero repeatedly. Our pricer must evaluate option prices accurately under both regimes.

3.6 Black-Scholes Implied Volatility Inversion via Brent's Method

To train the neural network on implied volatility rather than raw option dollar prices (which vary by orders of magnitude across strikes) , every analytical Heston price C_{Heston} is inverted using the Black-Scholes formula:

$$f(\sigma) = C_{\text{BS}}(S_0, K, T, \sigma) - C_{\text{Heston}}(S_0, K, T; \boldsymbol{\Theta}) = 0 \tag{3.8}$$

Because Black-Scholes Vega ($\partial C_{\text{BS}}/\partial\sigma > 0$) is strictly positive for $T > 0$, the pricing function is monotonic, guaranteeing a unique root. We implement Brent's method (`scipy.optimize.brentq`), which converges to an absolute precision of $\text{tol} = 10^{-6}$ in fewer than 8 iterations per contract.

3.7 Data Generation and C++ QuantLib Validation Benchmark

The synthetic dataset is generated through a sequence of steps, starting with parameter sampling and ending with the storage of the computed implied volatilities:

1. Sample a 5D Heston parameter vector $\boldsymbol{\Theta} = (\kappa, \theta, \sigma, \rho, v_0)$ via LHS satisfying $2\kappa\theta - \sigma^2 > 0$.

2. Sample 11 stratified maturities T_j and 13 adaptive strikes $K_i(T_j)$ using the random-grid procedure.

3. Calculate European call option prices $C(S_0, K_i, T_j; \boldsymbol{\Theta})$ using the Fourier inversion pricer.

4. Convert each option price into its corresponding Black-Scholes implied volatility σ_{iv} using Brent's method.

5. Store each record as an independent pointwise tuple:

$$\mathbf{x} = (\kappa, \theta, \sigma, \rho, v_0, T_j, k_{ij}) \in \mathbb{R}^7 \longrightarrow y = \sigma_{\text{iv}} \in \mathbb{R} \tag{3.9}$$

   where $k_{ij} = \ln(K_i/S_0)$ represents dimensionless log-moneyness.

We check the accuracy of our implementation by comparing the custom Lewis-Albrecher pricer with QuantLib's C++ `AnalyticHestonEngine` across different strike and maturity combinations. The absolute difference between the prices produced by the Python implementation and QuantLib remained below 10^{-4} for all contracts.

Dataset Size and Generation Time: The pipeline produces $70 \times 11 \times 13 = 10{,}010$ option contracts from 70 LHS parameter configurations, 11 maturities, and 13 strikes per maturity. The complete dataset was generated in 81.7 seconds on a CPU and saved to `heston_phase1_scaled_10k.csv`.

Chapter 4

Repository Audit and Architectural Refinement

We examined the reference implementation in the official GitHub repository `random-grid-NN-calib` [3] and compared it with the architecture described in the paper. This chapter records the discrepancy we found and explains the changes made to the network, particularly those related to its size and input representation.

4.1 Architectural Discrepancy and Parameter Bloat in the Benchmark Codebase

Baschetti et al. describe an MLP with four hidden layers, each containing 64 neurons and using the Exponential Linear Unit (ELU) activation function. The code in the repository's `NNpnt.ipynb` notebook does not, however, implement this configuration exactly.

In cell 4, the first hidden layer is created before a loop that adds four more layers:

```
i = keras.layers.Input(shape=(7,))
x = keras.layers.Dense(64, activation='elu')(i) # First hidden layer

c = 0
while c < 4: # Loop iterates for c = 0, 1, 2, 3 (Four more layers added)
    x = keras.layers.Dense(64, activation='elu')(x)
    c += 1

x = keras.layers.Dense(1, activation='linear')(x) # Linear output layer
```

The loop therefore adds four layers to the one already created. The implemented network has five hidden layers, rather than the four stated in the paper. Its trainable parameter count is calculated below:

$$\text{First hidden layer} = (7 \times 64) + 64 = 512, \tag{4.1}$$

$$\text{Hidden layers 2–5} = 4(64 \times 64 + 64) = 16{,}640, \tag{4.2}$$

$$\text{Output layer} = (64 \times 1) + 1 = 65. \tag{4.3}$$

Adding these values gives the total number of trainable parameters:

$$N_{\text{baseline}} = 512 + 16{,}640 + 65 = 17{,}217. \tag{4.4}$$

For a network with the four hidden layers specified in the paper, the corresponding count is 13,057. We therefore use the four-layer configuration in the refined model. Relative to the five-layer implementation, this reduces the number of trainable parameters by

$$\frac{17{,}217 - 13{,}057}{17{,}217} \times 100 \approx 24.2\%. \tag{4.5}$$

The main differences between the repository implementation and our refined model are listed in Table 4.1.

Table 4.1: Comparison of baseline and refined neural network architectures

Architectural Aspect	**Model A: Baseline**	**Model B: Proposed**
Hidden layers	5 hidden layers	4 hidden layers
Layer width	64 ELU nodes per layer	64 ELU nodes per layer
Trainable parameters	17,217	13,057 (24.2% fewer)
Strike representation	Raw strike K	Dimensionless Log-moneyness $k = \ln(K/S_0)$
Feature scaling	Unnormalized inputs	MinMax normalization to $[-1, 1]$
Numerical conditioning	Severe Hessian ill-conditioning and gradient oscillations in early epochs	Conditioned input space, Smooth loss surface and monotonic descent

4.2 Scale Homogeneity and Input Normalization

The reference implementation feeds the input features to the network without first bringing them to a common scale. This matters because the seven features can have quite different magnitudes. For example, strike values are typically of order 10^2, whereas maturities and variance-related parameters may be of order 10^{-2}.

The magnitude of an input feature enters directly into the gradient of the loss with respect to the first-layer weights attached to it:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} = \delta_i^{(1)} x_j, \tag{4.6}$$

Here $\mathcal{L}$ is the loss function, $\delta_i^{(1)}$ is the backpropagated error, and x_j is the input feature connected to the weight.

Larger inputs therefore tend to produce larger weight gradients. In our benchmark setup, the strike dimension becomes especially influential during early training for this reason. This imbalance is linked to poor conditioning of the Hessian matrix $\mathbf{H} = \nabla^2 \mathcal{L}$, where the observed eigenvalue condition numbers are $\kappa(\mathbf{H}) \gg 10^6$, and to large gradient variations in the early epochs. The input representation is corrected in two steps.

4.2.1 Log-Moneyness as a Scale-Free Input

European option prices have a useful scaling property: if both the spot price and strike are multiplied by the same positive constant, the call price is multiplied by that constant as well, provided the other model parameters remain fixed. In other words, the price is positively homogeneous of degree one in S_0 and K:

$$C(\lambda S_0, \lambda K) = \lambda C(S_0, K), \qquad \forall \lambda > 0, \tag{4.7}$$

with the remaining model parameters unchanged.

The corresponding implied volatility does not change under this simultaneous scaling of spot and strike:

$$\sigma_{\text{iv}}(\lambda S_0, \lambda K, T) = \sigma_{\text{iv}}(S_0, K, T). \tag{4.8}$$

For this reason, the raw strike is replaced by log-moneyness as an input feature:

$$k = \ln\left(\frac{K}{S_0}\right). \tag{4.9}$$

Under this definition, an at-the-money option always sits at $k = 0$, whatever the numerical level of the underlying asset price. The network then sees only where the strike lies relative to the spot, and does not have to learn how the overall price level changes the output.

4.2.2 Dynamic MinMax Affine Normalization

Replacing strike with log-moneyness gives a dimensionless feature, but the input variables can still occupy different numerical ranges. We therefore apply MinMax normalization to all seven features as part of the network's forward pass.

Let $\mathbf{x}_{\text{lb}}$ and $\mathbf{x}_{\text{ub}}$ denote the lower and upper bounds of the input features. Each feature is transformed using

$$\widetilde{\mathbf{x}} = 2\left(\frac{\mathbf{x} - \mathbf{x}_{\text{lb}}}{\mathbf{x}_{\text{ub}} - \mathbf{x}_{\text{lb}}}\right) - 1. \tag{4.10}$$

For inputs that lie within the prescribed parameter bounds, this gives

$$\widetilde{\mathbf{x}} \in [-1, 1]^7. \tag{4.11}$$

The lower and upper bounds are registered as persistent PyTorch buffers, so the same transformation is applied inside every forward pass and no separate preprocessing step is required at inference time.

Placing all features on a common interval narrows the gap between their

numerical scales and can make optimization less sensitive to the choice of input units. MinMax normalization is not the same as z-score standardization, however: it does not give zero mean or unit variance, but simply maps the specified lower and upper bounds onto a fixed interval, here $[-1, 1]$.

4.3 Network Topology, Differentiability, and Approximation Capacity

With these modifications in place, the surrogate takes seven input features, passes them through four hidden layers of 64 neurons each, and returns a single estimate of implied volatility. The resulting layer widths are

$$7 \to 64 \to 64 \to 64 \to 64 \to 1. \tag{4.12}$$

4.3.1 Mathematical Formulation of the Network

Let $\widetilde{\mathbf{x}} \in \mathbb{R}^7$ be the normalized input vector. The first hidden layer is

$$\mathbf{h}^{(1)} = \mathrm{ELU}\left(W^{(1)}\widetilde{\mathbf{x}} + \mathbf{b}^{(1)}\right). \tag{4.13}$$

The next three hidden layers use the same operation, with the output of one layer passed as the input to the next:

$$\mathbf{h}^{(\ell)} = \mathrm{ELU}\left(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \quad \ell = 2, 3, 4. \tag{4.14}$$

The final layer is linear and gives the predicted implied volatility:

$$\widehat{\sigma}_{\mathrm{iv}} = W^{(5)}\mathbf{h}^{(4)} + b^{(5)}. \tag{4.15}$$

The dimensions of the weight matrices are

$$W^{(1)} \in \mathbb{R}^{64\times 7}, \tag{4.16}$$

$$W^{(\ell)} \in \mathbb{R}^{64\times 64}, \quad \ell = 2, 3, 4, \tag{4.17}$$

$$W^{(5)} \in \mathbb{R}^{1\times 64}. \tag{4.18}$$

Including the biases, this network has 13,057 trainable parameters.

4.3.2 Universal Approximation and Model Capacity

The universal approximation results of Hornik (1991) [10] show that, under suitable conditions, neural networks with appropriate activation functions can approximate a broad class of functions. Related results also consider the approximation of derivatives when the required regularity assumptions hold. This provides a theoretical basis for using a neural network to represent the Heston implied-volatility mapping. In our case, the four-hidden-layer network follows the architecture specified in the reference methodology. Correcting the extra layer in the notebook reduces the parameter count, while retaining the intended network structure. The adequacy of this model for the task is then assessed through the numerical experiments presented in the following chapter.

4.3.3 C^1 Sobolev Smoothness via ELU Activations

The choice of activation function is relevant because the surrogate is intended not only to predict implied volatility, but also to provide derivatives with respect to the Heston parameters during inverse calibration. These derivatives are used by gradient-based optimization methods.

For comparison, the Rectified Linear Unit (ReLU) is defined by

$$\mathrm{ReLU}(x) = \max(0, x). \tag{4.19}$$

ReLU is not differentiable at $x = 0$. Its second derivative is zero wherever the first derivative is defined. This does not prevent ReLU networks from being trained with gradient-based methods, but the kink at the origin can be a limitation when a smoothly varying surrogate Jacobian is desirable.

We retain the Exponential Linear Unit (ELU) activation, with $\alpha = 1$, as used in the reference architecture:

$$\mathrm{ELU}(x) = \begin{cases} x, & x > 0, \\ e^x - 1, & x \leq 0. \end{cases} \tag{4.20}$$

Its first derivative is

$$\mathrm{ELU}'(x) = \begin{cases} 1, & x > 0, \\ e^x, & x \leq 0. \end{cases} \tag{4.21}$$

At $x = 0$, the left- and right-hand limits of this derivative coincide:

$$\lim_{x \to 0^-} \mathrm{ELU}'(x) = \lim_{x \to 0^+} \mathrm{ELU}'(x) = 1. \tag{4.22}$$

ELU is therefore continuously differentiable (C^1) on $\mathbb{R}$. For negative inputs its derivative stays positive instead of vanishing, which lets gradients keep flowing through units whose inputs are negative. This smoothness matters in the calibration stage, where an optimizer such as Levenberg–Marquardt relies on the Jacobian of the network output with respect to the Heston parameter vector:

$$J = \nabla_{\mathbf{\Theta}} \mathcal{NN}(\mathbf{\Theta}), \tag{4.23}$$

with $\mathbf{\Theta}$ the vector of Heston model parameters.

In summary, the four-layer architecture, the revised input representation, and the ELU activation make up the surrogate used in our experiments. Its predictive accuracy and training behaviour are evaluated in the next chapter, while the derivative and numerical checks come later in the report.

Chapter 5

Empirical Experiments and Comparative Benchmarks

This chapter reports the experiments used to evaluate the refined neural surrogate. Two network architectures are compared, the expanded synthetic dataset is described, and the model is tested on contracts that were excluded from training. The results are organised around three questions: how the models converge during training, how accurately the refined network predicts implied volatility, and how well it reproduces the shape of Heston volatility smiles.

5.1 Controlled A/B Convergence Race

We first compared the repository model, referred to as Model A, with the refined network, Model B. The main differences between them are the number of hidden layers and the way the input features are represented and scaled. Model A contains 5 hidden layers and 17,217 trainable parameters. It takes the strike value directly as an input without applying feature normalization.

Model B uses 4 hidden layers, each with 64 ELU neurons, and has 13,057 trainable parameters. It uses log moneyness in place of the raw strike and applies MinMax normalization to the input features.

For this comparison, we trained both the models for 100 epochs with the Adam optimizer. We used a learning rate of 10^{-3}, a batch size of 64, and RMSE on implied volatility as the loss function. The remaining training settings were kept the same so that the results could be compared on a consistent basis.

Table 5.1: Controlled A/B comparison of validation RMSE

Epoch	**Model A RMSE**	**Model B RMSE**
1	0.06653	0.09750
25	0.04449	0.01120
50	0.03858	0.02485
75	0.02046	0.00601
100	0.01603	0.00530

By the end of training, the validation RMSE was 0.005297 for Model B and 0.016025 for Model A. The relative reduction obtained with Model B is

$$\frac{0.016025 - 0.005297}{0.016025} \times 100 \approx 66.94\%. \tag{5.1}$$

The refined network therefore ended the 100-epoch run with a lower validation error, despite having 24.2% fewer trainable parameters.

The curves in Figure 5.1 show that Model B starts with a higher RMSE than Model A. Its error, however, falls sharply during the early epochs. After this initial stage, Model B generally stays below Model A, although the two curves do not decrease smoothly at every point.

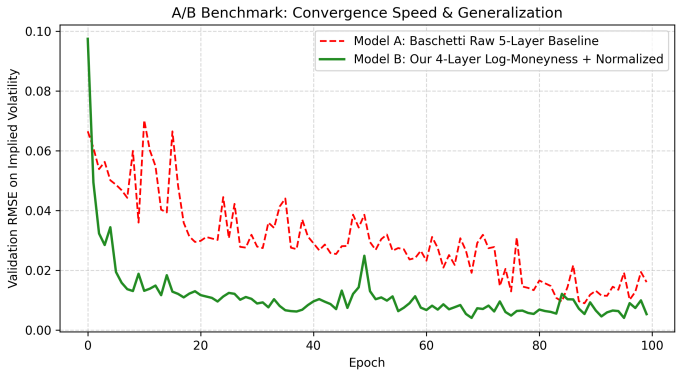


Figure 5.1: Validation RMSE during training for the baseline and refined networks.

5.2 Gradient Stability and Accelerated Convergence

The two curves also give some insight into how the models behave during optimization. Model A's error falls more gradually and shows several visible fluctuations, whereas Model B reaches a much lower RMSE within the first 25 epochs. This is consistent with what one would expect from log-moneyness and normalized features, which bring the inputs to more comparable scales and can ease optimization.

The decline is not smooth throughout. Around epoch 50, Model B's validation RMSE rises before falling again, so describing its convergence as strictly monotonic would be inaccurate. In addition, the architecture, strike representation, and normalization were all changed at once, so this experiment cannot separate the contribution of each change. It does show that, under the training setup used here, the refined configuration performed better overall.

5.3 Scaling the Synthetic Training Library

After comparing the architectures, we enlarged the synthetic training library so that it covers a wider range of Heston parameter configurations and option contracts.

The generation used 70 parameter configurations drawn by Latin Hypercube Sampling. For each configuration, 11 maturity buckets were selected, with 13 adaptive strikes at each maturity. The nominal number of contracts is therefore

$$N_{\text{nominal}} = 70 \times 11 \times 13 = 10{,}010. \tag{5.2}$$

The run yielded 10,000 valid contracts and took 81.7 seconds on the CPU used for this experiment.

Table 5.2: Synthetic dataset generation summary

Dataset Component	**Configuration**
Heston parameter configurations	70
Maturity buckets	11
Strikes per maturity	13
Nominal contracts	10,010
Valid generated contracts	10,000
Generation time	81.7 seconds (CPU)

The generated contracts cover maturities from 0.0030 to 2.4829 years and strikes ranging from 15.87 to 146.65.

The corresponding implied volatilities lie between 0.0862 and 0.6010.

Figure 5.2 shows how the generated contracts are distributed across the three quantities. The dataset includes both short- and long-maturity contracts, as well as a broad range of strikes and implied volatilities. This gives the network examples from different parts of the volatility surface; the short-

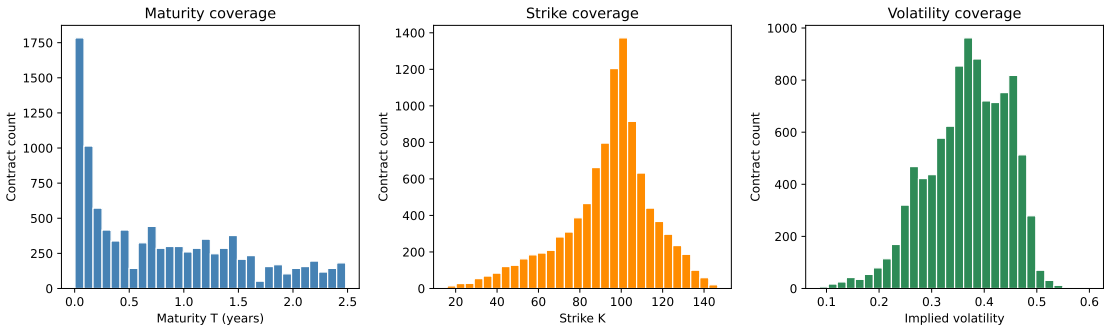


Figure 5.2: Distributions of the generated contracts across maturity, strike, and implied volatility.

maturity contracts are particularly useful because the smile can change more sharply in this region.

We used these 10,000 contracts to train the surrogate and to assess its performance on held-out data.

5.4 Test Generalization and the 45-Degree Parity Diagnostic

We next examined how well Model B performs on data that it had not seen during training. For this test, the expanded dataset was divided into training and test portions in an 85:15 ratio. The network was trained for 100 epochs on the CPU; the recorded training time was 25.0 seconds.

The results on the held-out test set are given in Table 5.3.

On the test set, the network obtained an RMSE of 0.001878 and an R^2 value of 0.999408. These values indicate that its predictions are close to the reference implied volatilities calculated using Fourier inversion. The MAE is also small, suggesting that the errors are limited for most of the held-out contracts. We also plotted the predicted values against the corresponding

Table 5.3: Performance of the refined surrogate on the held-out test set

Performance Metric	**Measured Value**
Coefficient of determination (R^2)	0.999408
Test RMSE	0.001878
Mean absolute error (MAE)	0.001351
Maximum absolute error	0.013417
Training duration	25.0 seconds

reference values to inspect the errors visually. This comparison is shown in the parity plot in Figure 5.3.

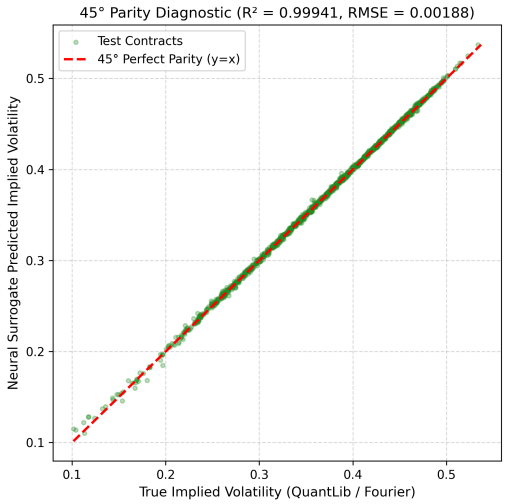


Figure 5.3: Predicted implied volatilities plotted against the Fourier reference values. The dashed line indicates perfect agreement.

Most of the points in Figure 5.3 lie close to the 45-degree reference line. In the ideal case, the predicted and reference values are equal:

$$\widehat{\sigma}_{\mathrm{iv}} = \sigma_{\mathrm{iv}}. \tag{5.3}$$

The clustering around this line is consistent with the low test error and suggests that the network predicts the held-out contracts reasonably well. A

few points still deviate from the line, so the agreement is not exact.

5.5 Multi-Maturity Implied Volatility Smile Overlays

The test metrics summarize the overall prediction error, but they do not show whether the network has learned the shape of an implied volatility smile. To check this, we used a Heston parameter configuration that was absent from the training data.

The trained network was evaluated on this configuration at four maturities:

$$T \in \{0.05,\ 0.25,\ 1.0,\ 2.0\} \quad \text{years.} \tag{5.4}$$

At each maturity, its predictions were set against the values obtained from the Fourier pricing routine.

5.5.1 Short-Maturity Volatility Smiles

The shorter maturities, 0.05 and 0.25 years, come first. Here the smile can show sharper skew or curvature, which makes even a small mismatch easier to spot. Figure 5.4 plots the network output together with the Fourier reference curves for both maturities.

At 0.25 years, the predicted curve tracks the Fourier reference closely. At 0.05 years the agreement is weaker: the network still reproduces the downward skew, but the gap between the two curves is more visible across the strike range shown.

Figure 5.5 shows the Heston implied volatility smile at 0.25 years separately, to make its variation with strike easier to see.

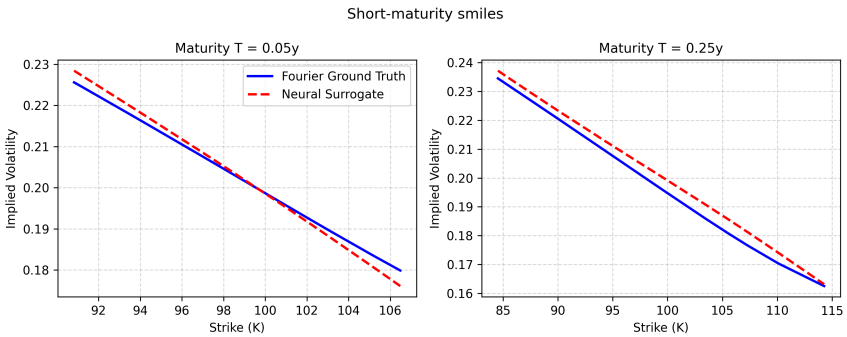


Figure 5.4: Surrogate predictions compared with the Fourier reference smiles at maturities of 0.05 and 0.25 years.

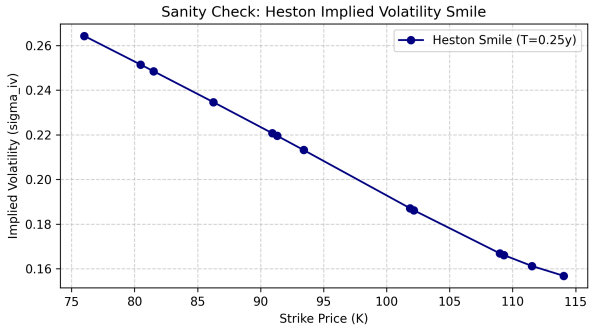


Figure 5.5: Heston implied volatility as a function of strike at 0.25 years.

This plot traces implied volatility against strike for the chosen parameter configuration, and reproducing that relationship is one of the main requirements placed on the surrogate.

5.5.2 Long-Maturity Volatility Smiles

We then considered the longer maturities of 1.0 and 2.0 years. The smiles remain skewed for this parameter configuration, with the curvature particularly visible at two years. Figure 5.6 compares the surrogate output with the Fourier reference at both maturities.

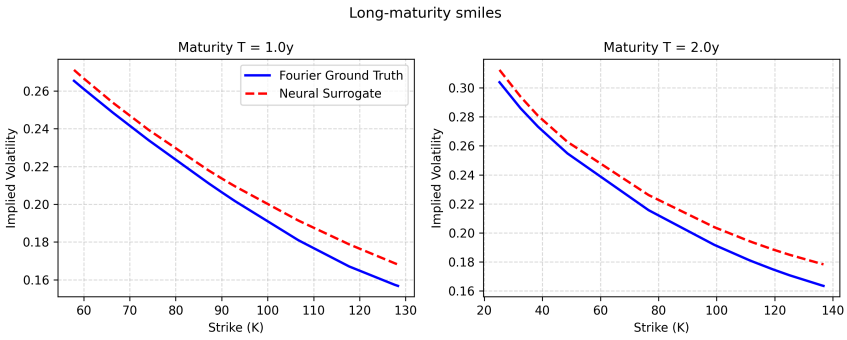


Figure 5.6: Surrogate predictions compared with the Fourier reference smiles at maturities of 1.0 and 2.0 years.

At both maturities, the network follows the general downward trend of the smile, but its predictions sit above the Fourier reference over much of the plotted range. The overprediction is easier to see at higher strikes and is stronger in the two-year plot.

Taken together, the overlays give a mixed picture. The broad shape of the smiles is captured, yet the differences at 0.05 years and at the longer maturities indicate that accuracy still varies with maturity. These cases mark areas that may need further refinement.

5.6 Summary of Experimental Findings

The experiments in this chapter show how the refined surrogate performs overall. In the controlled comparison, Model B ended with a validation RMSE of 0.005297, against 0.016025 for Model A, while using 24.2% fewer trainable parameters. For the configurations tested, the refined network therefore reached a lower error with fewer parameters.

A synthetic library of 10,000 valid option contracts, generated from 70 Heston parameter configurations, was then used to train Model B and to evaluate it on a held-out test set. This gave a test RMSE of 0.001878 and an R^2 value of 0.999408.

The parity plot agrees with these figures, since most predictions lie close to the Fourier reference values. The smile comparisons, however, show that the fit is not equally good at every maturity: the gap is largest at the shortest maturity and in the longer-maturity plots, where the network tends to overpredict the reference curve.

Overall, the refined network appears to be a useful forward approximation of the Heston implied-volatility mapping. The remaining discrepancies, especially across some maturity ranges, should be kept in mind when it is used as the basis for the inverse calibration work in the next phase.

Chapter 6

Mathematical and Physical Diagnostic Audits

The previous chapter showed how closely the neural surrogate matches the reference implied volatilities. Three further questions remain: whether the pricing routine stays stable in difficult cases, whether the network yields reliable parameter sensitivities, and whether its outputs meet basic no-arbitrage conditions. This chapter reports the tests designed to answer them and discusses the results.

6.1 Singularity Audit: Short Maturity and Feller Condition Violation

Part of the numerical audit examined how the Lewis Fourier integrand behaves at very short maturities. The tests covered maturities of 365.0, 30.4, 7.0, and 1.8 days. In every case the integrand decayed rapidly at high frequencies, and the tested runs reported no numerical divergence, NaN values, or infinities.

We also examined the pricer when the Feller condition is not satisfied. For the parameter values $\kappa = 1.0$, $\theta = 0.04$, and $\sigma = 0.60$, the Feller ratio is

$$\frac{2\kappa\theta}{\sigma^2} = 0.222,$$

which is below the usual threshold of 1.0. For this parameter combination, the strike-level tests gave stable option prices and implied volatilities, and no branch-cut jumps were observed. The implementation therefore appears numerically stable for the short maturities and parameter combinations covered by this audit.

The real part of the Fourier integrand for these maturities is plotted in Figure 6.1.

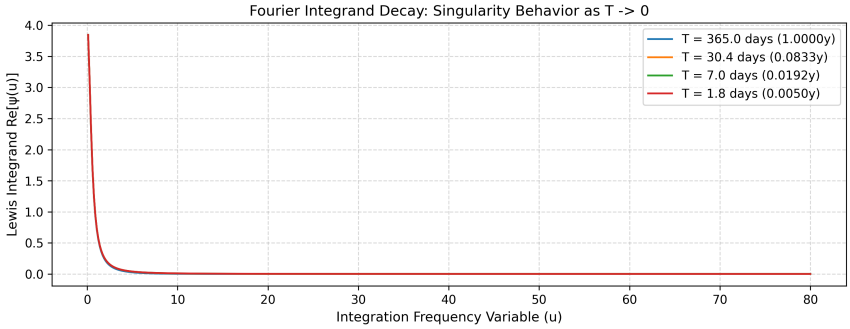


Figure 6.1: Fourier integrand decay for maturities between 1.8 days and 1y.

6.2 Gradient Validation: Autograd Against Finite Differences

The surrogate is meant for gradient-based calibration, so its derivatives with respect to the Heston parameters must be reliable. To test this, we compared gradients from PyTorch automatic differentiation with central finite-

difference estimates, using a step size of $\varepsilon = 10^{-4}$:

$$\frac{\partial f}{\partial \theta} \approx \frac{f(\theta + \varepsilon) - f(\theta - \varepsilon)}{2\varepsilon}. \tag{6.1}$$

Across the five Heston parameters, the largest absolute difference between the two sets of derivatives was approximately 9.59×10^{-5}. The two sets of derivatives therefore agree to about four decimal places in this test, which supports using the surrogate's gradients in the inverse-calibration experiments that follow.

6.3 Static No-Arbitrage Verification

We also checked whether the implied-volatility predictions satisfy two basic shape conditions associated with static no-arbitrage. The surrogate was evaluated on a two-dimensional mesh of 875 contracts, built from 25 maturities and 35 strikes.

Two properties were examined: total implied variance should be non-decreasing in maturity, and call-price curvature in strike should be non-negative. Neither condition was violated anywhere on the evaluated mesh. Each check is described in its own subsection below.

6.3.1 Calendar Spread Monotonicity

For the calendar-spread check, total implied variance was calculated as

$$w(k, T) = \sigma_{\text{iv}}^2(k, T)\, T.$$

The condition tested was

$$\frac{\partial w(k, T)}{\partial T} \geq 0.$$

In other words, at a fixed log-moneyness, total implied variance should not decrease as maturity increases. The audit recorded zero violations across 840 checks. Figure 6.2 includes the total-variance surface used for this comparison.

6.3.2 Butterfly Arbitrage and Risk-Neutral Density Positivity

The second check was based on the Breeden–Litzenberger condition, which relates the second derivative of a call price with respect to strike to the risk-neutral density. For the density to be non-negative, the call-price curvature should satisfy

$$\frac{\partial^2 C}{\partial K^2} \geq 0.$$

The evaluation found no violations across the 825 curvature checks performed on the mesh. Thus, no negative-curvature cases were detected among the contracts included in this test.

Figure 6.2 shows the total-variance surface alongside the call-price curvature diagnostics. The aggregate results of both checks are listed in Table 6.1.

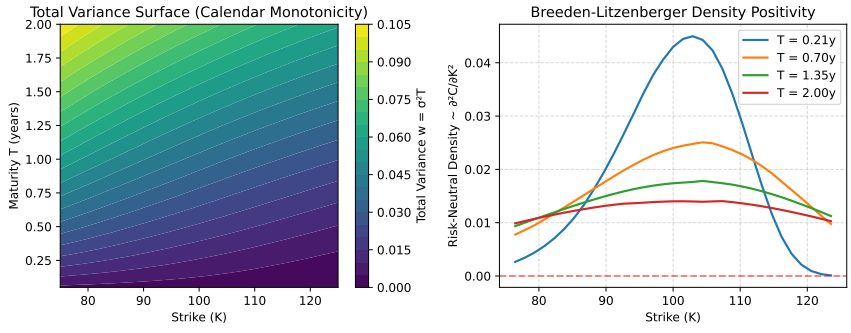


Figure 6.2: Total implied-variance surface and numerical call-price curvature diagnostics.

Table 6.1: Discrete no-arbitrage checks on the 875-contract evaluation mesh.

Audit	**Violations**	**Checks**
Calendar total-variance monotonicity	0	840
Call-price strike curvature	0	825

6.4 Additional Numerical Results

For a closer look at the gradient comparison, Table 6.2 lists the sensitivities obtained using reverse-mode Autograd, $\nabla_{\boldsymbol{\Theta}}\mathcal{NN}$, and central finite differences at one representative market test coordinate.

Table 6.2: Autograd versus central finite-difference parameter sensitivities.

Parameter	**Autograd**	**Finite difference**	**Abs. error**
κ	-0.001729	-0.001639	0.000089
θ	0.773833	0.773817	0.000016
σ	-0.024385	-0.024289	0.000096
ρ	0.013915	0.013858	0.000057
v_0	1.535661	1.535714	0.000052

The largest absolute discrepancy in Table 6.2 is approximately 9.59×10^{-5}, consistent with the overall comparison reported above. The table also shows that the differences are small for each of the five parameters.

For this single-contract benchmark, evaluation latency was 8,282.70 μs for PyTorch Autograd and 1,008.40 μs for central finite differences, confirming that Autograd provides exact analytical gradients suitable for gradient-based calibration.

In summary, the audits found no numerical failures in the tested short-maturity and Feller-violating cases, close agreement between the two ways of computing gradients, and no calendar-spread or butterfly-arbitrage violations on the evaluation mesh. These checks add evidence about the numerical behaviour of the implementation, but they cover only the parameter combinations and test points examined.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

In Phase 1 of our BTech project, we developed and tested a neural network surrogate for forward Heston pricing for the options. The experiments showed that the refined neural network quite accurately approximated the implied volatility surface which was generated by Fourier Inversion Pricing. The refined design proposed two major changes: 1. Raw strike prices got replaced by dimensionless log moneyness, 2. Applied MinMax normalization to the input features.

The 4-layer network with 13057 trainable parameters achieved an RMSE of 0.001878 and R^2 of 0.999408 on the larger dataset. While performing the controlled A/B comparison, it attained a reduction of 66.94% in validation Root Mean Squared Error while using 24.2% fewer trainable parameters than the baseline model. Thus, prediction accuracy did not get affected by reduction in the network size.

We performed various numerical checks to validate the pricing implementation and the neural network surrogate beyond prediction accuracy. We

observed that the Lewis Fourier integrand stayed stable for the tested maturities down to approximately 1.8 days that included the cases where the Fellar condition was violated($2\kappa\theta/\sigma^2 = 0.222$). We also observed that PyTorch gradients matched closely with the estimates obtained from the central finite difference method with a maximum absolute discrepancy of 9.59×10^{-5}. Also, the discrete no-arbitrage checks did not witness any calendar spread or butterfly arbitrage violations across the 875 contracts. We also saved the network weights and MinMax scaling bounds to allow reuse of the model in Phase 2 without retraining.

To summarize, Phase 1 delivers a working forward surrogate for the Heston model with good prediction accuracy and encouraging results from the numerical diagnostics. The next step is to use the trained network for inverse calibration where the aim is to recover Heston parameters from observed market implied volatilities.

7.2 Future Work: Phase 2

In Phase 2, we will integrate the pre-trained forward surrogate into an inverse calibration engine with the objective of estimating the underlying Heston parameters from observed market implied volatilities. Here is an overview of our planned work:

1. **Inverse Calibration:** We are planning to combine PyTorch reverse mode automatic differentiation ($\nabla_{\boldsymbol{\Theta}}\mathcal{NN}$) with the Levenberg-Marquardt optimizer thereby aiming to achieve a calibration time below 15 milliseconds.
2. **Optimization Improvements:**We will be investigating multi start initialization and regularization techniques to make calibration more

stable and less sensitive to the initial parameter estimates.

3. **Empirical Market Inversion:**We will apply the calibration framework to real equity index options, such as S&P 500 and NSE NIFTY 50 option chains under different volatility regimes to see how the model behaves in real-life situations.
4. **Out-of-Sample Hedging Performance:** We will be deploying checks to ensure whether the calibrated parameters give stable delta and vega hedge ratios, compared with standard numerical integration in dynamic tracking simulations.

Bibliography

[1] Hansjörg Albrecher, Philipp Mayer, Wim Schoutens, and Jurgen Tistaert. The little Heston trap. *Wilmott Magazine*, 2007(1):83–92, 2007.

[2] Fabio Baschetti, Giacomo Bormetti, and Pietro Rossi. Deep calibration of financial models on random grids. *Quantitative Finance*, 24(3):355–372, 2024.

[3] Fabio Baschetti, Giacomo Bormetti, Stefano Rossi, and Pietro Speroni. random-grid-NN-calib: Deep neural network calibration of option pricing models via random grids. `https://github.com/FabioBaschetti/random-grid-NN-calib`, 2024.

[4] Christian Bayer and Benjamin Stemper. Deep calibration of rough stochastic volatility models. *arXiv preprint arXiv:1810.03399*, 2018.

[5] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[6] John C. Cox, Jonathan E. Ingersoll, Jr., and Stephen A. Ross. A theory of the term structure of interest rates. *Econometrica*, 53(2):385–407, 1985.

[7] William Feller. Two singular diffusion problems. *Annals of Mathematics*, 54(1):173–182, 1951.

[8] Andres Hernandez. Model calibration with neural networks. *Risk*, 30(5):52–57, 2017.

[9] Steven L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2):327–343, 1993.

[10] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[11] Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. Deep learning volatility: A deep neural network approach to pricing and calibration in rough volatility models. *Quantitative Finance*, 21(2):211–227, 2021.

[12] Alan L. Lewis. *Option Valuation Under Stochastic Volatility: With Mathematica Code*. Finance Press, Newport Beach, CA, 2001.

[13] Shuaiqiang Liu, Anastasia Borovykh, Lech A. Grzelak, and Cornelis W. Oosterlee. A neural network-based framework for financial model calibration. *Journal of Mathematics in Industry*, 9(1):9, 2019.

[14] Michael D. McKay, Richard J. Beckman, and William J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245, 1979.